

DESIGNUX.md — Guide des bonnes pratiques UX/UI appliquées

Document de référence pour le portfolio de Djefrid Byli. Ce fichier détaille toutes les décisions UX/UI prises, les outils utilisés et la logique derrière chaque choix. Il peut être réutilisé comme base sur d'autres projets.

Sommaire

- 1. [Stack UX/UI utilisée](#)
- 2. [Design System — Tokens & Classes globales](#)
- 3. [Thème sombre / clair \(Dark Mode\)](#)
- 4. [Animations avec Framer Motion](#)
- 5. [Navigation & Header](#)
- 6. [Menu mobile — Bonnes pratiques](#)
- 7. [Section Hero](#)
- 8. [Section About — Accordion "Lire la suite"](#)
- 9. [Section Projects — Carousel + Modal](#)
- 10. [Section Skills — Carousel horizontal](#)
- 11. [Section Contact — Formulaire accessible](#)
- 12. [Skeleton Loaders \(états de chargement\)](#)
- 13. [Page 404 personnalisée](#)
- 14. [SEO & Métadonnées](#)
- 15. [Accessibilité \(a11y\)](#)
- 16. [Performance](#)
- 17. [Internationalisation FR/EN](#)
- 18. [Anti-spam formulaire \(Honey,pot\)](#)

1. Stack UX/UI utilisée

Outil	Rôle	Version
Next.js 14 (App Router)	Framework React, SSR, routing	14.x
Tailwind CSS	Styling utility-first	3.x
Framer Motion	Animations déclaratives React	dernière
next-themes	Gestion dark/light mode sans flash	dernière
shadcn/ui	Composants UI accessibles (Input, Textarea, Label, Badge)	dernière
clsx	Construction conditionnelle de classes CSS	dernière
lucide-react	Icônes SVG cohérentes et légères	dernière
Vercel Analytics	Suivi des performances UX réelles	dernière

2. Design System — Tokens & Classes globales

Palette de couleurs (tailwind.config)

- **Couleur primaire** : primary-400 à primary-700 (indigo/violet — #6366f1)
- **Fond sombre** : dark-950 → dark-700 (échelle de gris très sombres)
- **Texte** : text-white , text-gray-100 à text-gray-500
- **Statut** : green-400 (disponible), yellow-500 (avertissement), red-400 (erreur)

Classes composants réutilisables (globals.css)

```
/* Conteneur centré responsive */
.section-container → max-w-6xl mx-auto px-4 sm:px-6 lg:px-8

/* Typographie de section */
.section-title → text-3xl sm:text-4xl font-bold text-white mb-4
.section-subtitle → text-gray-400 text-lg mb-12

/* Boutons */
.btn-primary → bg-primary-600, hover:bg-primary-700, focus:ring-4
.btn-secondary → border border-primary-500, hover:bg-primary-500/10

/* Cartes */
.card → bg-dark-900 border border-dark-800 rounded-xl p-6 hover:border-
primary-500/50

/* Tags compétences */
.skill-tag → px-3 py-1.5 bg-dark-800 rounded-lg, hover:bg-primary-500/20

/* Anti-spam caché */
.honeypot-field → position absolute, left -9999px (invisible)
```

Pourquoi ce système ? Définir des classes réutilisables dans `@layer components` assure la cohérence visuelle sur tout le site sans répéter du Tailwind. Un changement dans `globals.css` se répercute partout.

3. Thème sombre / clair (Dark Mode)

Outil : `next-themes` + **overrides CSS manuels**

Fichier : [components/ui/ThemeToggle.tsx](#) + [app/globals.css](#)

Pourquoi `next-themes` ?

- Évite le **flash blanc au chargement** (FOUC — Flash Of Unstyled Content)
- Le thème est persisté dans `localStorage` automatiquement
- `suppressHydrationWarning` sur `<html>` évite les erreurs React SSR

Configuration

```
// app/layout.tsx
<html suppressHydrationWarning>
```

```
// components/Providers.tsx
<ThemeProvider attribute="class" defaultTheme="dark" enableSystem={false}>
```

- `attribute="class"` : ajoute/retire la classe `.dark` sur `<html>`
- `defaultTheme="dark"` : le site s'ouvre **toujours en sombre**
- `enableSystem={false}` : ignore les préférences système — cohérence garantie

ThemeToggle — Pattern "montage conditionnel"

```
const [mounted, setMounted] = useState(false);
useEffect(() => setMounted(true), []);
if (!mounted) return <div className="w-9 h-9" />; // espace réservé
```

Ce pattern évite le **mismatch d'hydratation SSR** : le serveur ne connaît pas le thème stocké dans le navigateur. On affiche un placeholder de même taille pour éviter le layout shift (CLS).

Animation icône Lune/Soleil

```
<AnimatePresence mode="wait" initial={false}>
  {isDark ? (
    <motion.span
      key="moon"
      initial={{ rotate: -90, opacity: 0, scale: 0.5 }}
      animate={{ rotate: 0, opacity: 1, scale: 1 }}
      exit={{ rotate: 90, opacity: 0, scale: 0.5 }}
      transition={{ duration: 0.25, ease: "easeInOut" }}
    />
  ) : ...}
</AnimatePresence>
```

- `mode="wait"` : attend que l'icône précédente finisse de sortir avant d'entrer la suivante
- Rotation + scale + opacité = transition fluide et perceptible sans être agressive

Mode clair — Overrides CSS

Le design est dark-first. Pour le mode clair, on surcharge via `html:not(.dark)` dans `globals.css` :

- Fonds : blanc pur, gris très légers
- Textes : slate foncés pour contraste WCAG AA
- Cartes : ombres portées subtiles (box-shadow) pour compenser l'absence de contraste de fond
- Bordures : slate légers
- Sections alternées : Hero/Projets/Contact en blanc, About/Skills en gris très léger (`#f8f9fc` , `#f1f5f9`)

4. Animations avec Framer Motion

4.1 FadeInSection — Animation au scroll

Fichier : [components/ui/FadeInSection.tsx](#)

```
const ref = useRef(null);
const isInView = useInView(ref, { once: false, margin: "-80px" });

<motion.div
  initial={{ opacity: 0, y: 30 }}
  animate={isInView ? { opacity: 1, y: 0 } : { opacity: 0, y: 30 }}
  transition={{ duration: 0.6, delay, ease: "easeOut" }}
>
```

- `useInView` de Framer Motion surveille l'intersection avec le viewport
- `margin: "-80px"` : déclenche l'animation 80px avant l'entrée dans l'écran
- `once: false` : rejoue l'animation si on rescroll vers le haut (choix intentionnel)
- `delay` paramétrable pour cascader les éléments d'une même section

Usage type dans chaque section :

```
<FadeInSection>                {/* titre */}
<FadeInSection delay={0.1}>    {/* premier bloc */}
<FadeInSection delay={0.2}>    {/* deuxième bloc */}
```

4.2 Hero — Entrée séquentielle

```
// Badge : delay 0
// H1 (nom) : delay 0.1
// H2 (titre) : delay 0.2
// Localisation : delay 0.3
// Boutons : delay 0.4
```

Chaque élément apparaît 100ms après le précédent → effet de cascade naturel qui guide l'œil de haut en bas.

4.3 ProjectCard — Hover Spring

```
<motion.article
  whileHover={{
    y: -6,
    boxShadow: "0 0 24px rgba(99, 102, 241, 0.25)",
    borderColor: "rgba(99, 102, 241, 0.5)",
  }}
  transition={{ type: "spring", stiffness: 300, damping: 20 }}
>
```

- `type: "spring"` : rebond physique réaliste (vs `ease` linéaire trop mécanique)
- `stiffness: 300, damping: 20` : ressort rapide mais sans oscillation excessive
- `y: -6` : élévation = feedback que la carte est cliquable
- Glow indigo : cohérent avec la couleur primaire

4.4 AnimatePresence — Menu mobile & icône thème

`AnimatePresence` permet d'animer la **sortie** d'éléments retirés du DOM (ce que React ne gère pas nativement) :

```
<AnimatePresence>
  {isOpen && <motion.div exit={{ opacity: 0, y: -8 }} />}
</AnimatePresence>
```

5. Navigation & Header

Fichier : [components/Header.tsx](#)

Navbar transparente → opaque au scroll

```
useEffect(() => {
  const handleScroll = () => setIsScrolled(window.scrollY > 50);
  window.addEventListener("scroll", handleScroll);
  return () => window.removeEventListener("scroll", handleScroll);
}, []);

className={isScrolled
  ? "bg-dark-950/95 backdrop-blur-md shadow-lg"
  : "bg-transparent"}
```

- Transparent au sommet : le Hero s'affiche sans barrière visuelle
- Dès 50px de scroll : fond semi-opaque + `backdrop-blur-md` (effet verre dépoli)
- `transition-all duration-300` : fondu doux entre les deux états

Lien actif — IntersectionObserver

```
const observer = new IntersectionObserver(
  (entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) setActiveSection(entry.target.id);
    });
  },
  { rootMargin: "-40% 0px -55% 0px", threshold: 0 }
);
```

- `rootMargin: "-40% 0px -55% 0px"` : seule la zone centrale du viewport (5%) déclenche le changement
- Résultat : le lien actif change uniquement quand la section est vraiment "au centre" de l'écran
- Indicateur visuel : underline animée + couleur `text-primary-400`

Logo dynamique

```
{profile.name || "Portfolio"}
```

Le nom est chargé depuis Firebase — pas de valeur hardcodée.

6. Menu mobile — Bonnes pratiques

Fichier : [components/Header.tsx](#) (lignes 51–248)

Problème résolu

Sur mobile, sans protection : le contenu derrière le menu reste scrollable et visible → expérience confuse.

Solution appliquée : 3 mécanismes

1. Scroll lock

```
useEffect(() => {  
  document.body.style.overflow = isMobileMenuOpen ? "hidden" : "";  
  return () => { document.body.style.overflow = ""; };  
}, [isMobileMenuOpen]);
```

2. Fermeture par touche Escape

```
const handleKeyDown = (e: KeyboardEvent) => {  
  if (e.key === "Escape") closeMobileMenu();  
};  
if (isMobileMenuOpen) document.addEventListener("keydown", handleKeyDown);
```

3. Backdrop semi-transparent + fermeture au clic extérieur

```
<motion.div  
  className="fixed inset-0 z-30 md:hidden bg-black/60"  
  onClick={closeMobileMenu}  
  aria-hidden="true"  
>
```

Structure des z-index

```
z-50 → Header fixe (toujours au-dessus)  
z-40 → Dropdown menu compact (fixed top-16)  
z-30 → Backdrop (fixed inset-0)
```

Animation d'entrée/sortie

```
<motion.div  
  initial={{ opacity: 0, y: -8 }}  
  animate={{ opacity: 1, y: 0 }}  
  exit={{ opacity: 0, y: -8 }}  
  transition={{ duration: 0.18, ease: "easeOut" }}  
>
```

Durée courte (180ms) pour une réponse snappy sur mobile.

Accessibilité bouton hamburger

```
aria-expanded={isMobileMenuOpen ? "true" : "false"}
aria-label="Menu de navigation"
className="focus:ring-2 focus:ring-primary-500 rounded-lg"
```

7. Section Hero

Fichier : [components/sections/Hero.tsx](#)

Badge "Open to Work" pulsé

```
<span className="animate-ping absolute inline-flex h-full w-full rounded-full bg-green-400
opacity-75" />
<span className="relative inline-flex rounded-full h-2 w-2 bg-green-400" />
```

Pattern Tailwind "ping" : deux cercles superposés, l'un s'agrandit en fadeout, l'autre reste fixe. Attire l'attention sans être agressif. Conditionnel : ne s'affiche que si `profile.openToWork === true` .

Glow background

```
<div className="absolute top-1/3 left-1/2 -translate-x-1/2 w-[600px] h-[400px] bg-primary-
600/20 rounded-full blur-[120px]" />
```

- Élément décoratif `pointer-events-none` (n'interfère pas avec les clics)
- `blur-[120px]` sur un cercle coloré = effet "glow" douce ambiance
- Deux blobs de tailles différentes pour la profondeur

Scroll indicator animé

```
<div className="animate-bounce hidden md:block">
  <Link href="#about" aria-label={t('hero.scrollDown')}>
    ↓
  </Link>
</div>
```

- `hidden md:block` : visible seulement sur desktop (trop distrayant sur mobile)
- `animate-bounce` Tailwind : animation CSS pure, zéro JS
- `aria-label` traduit : accessible aux lecteurs d'écran

Boutons CTA responsive

```
className="flex flex-col sm:flex-row items-center justify-center gap-4"
```

Empilés verticalement sur mobile (`flex-col`), côte à côte sur tablette+ (`sm:flex-row`).

8. Section About — Accordion "Lire la suite"

Fichier : [components/sections/About.tsx](#)

Pattern d'accordion CSS (sans bibliothèque)

```
<div className={`overflow-hidden transition-all duration-500 ease-in-out ${
  expanded ? "max-h-[2000px]" : "max-h-[12rem]"
}`}>
```

- `max-h` + `overflow-hidden` + `transition-all` = animation CSS pure, pas de JS pour la hauteur
- `max-h-[2000px]` au lieu de `max-h-none` car CSS ne peut pas transitionner vers `none`
- Durée 500ms pour une ouverture fluide (pas brusque)

Dégradé masquant

```
{!expanded && (
  <div className="absolute bottom-0 left-0 right-0 h-16 bg-gradient-to-t from-dark-900 to-transparent pointer-events-none" />
)}
```

Signale visuellement qu'il y a plus de contenu → invite au clic sans forcer.

Bouton avec flèche directionnelle

Flèche vers le bas = "il y a plus à voir", flèche vers le haut = "réduire". Feedback immédiat sans texte ambigu.

9. Section Projects — Carousel + Modal

Fichier : [components/sections/Projects.tsx](#)

Carousel CSS snap (sans bibliothèque)

```
<div className="flex gap-6 overflow-x-auto snap-x snap-mandatory pb-2 scrollbar-none">
  <div className="flex-shrink-0 w-full sm:w-[calc(50%-12px)] lg:w-[calc(33.33%-16px)] snap-start">
```

- `snap-x snap-mandatory` : le scroll "claque" sur chaque carte (effet natif CSS)
- `scrollbar-none` : masque la barre de scroll (`scrollbar-width: none` + `::-webkit-scrollbar`)
- Largeurs responsive : 1 carte / 2 cartes / 3 cartes selon la taille d'écran

Navigation par flèches

```
const scroll = (dir: "prev" | "next") => {
  const cardWidth = card ? card.offsetWidth + 24 : 320; // 24px = gap-6
  el.scrollBy({ left: dir === "next" ? cardWidth : -cardWidth, behavior: "smooth" });
};
```

Calcule la largeur réelle de la première carte (responsive) pour scroller exactement d'une carte.

Modal détail projet — Bonnes pratiques

```
// 1. Scroll lock
useEffect(() => {
  document.body.style.overflow = "hidden";
  return () => { document.body.style.overflow = ""; };
}, []);

// 2. Fermeture Escape
useEffect(() => {
  const handler = (e: KeyboardEvent) => { if (e.key === "Escape") onClose(); };
  window.addEventListener("keydown", handler);
  return () => window.removeEventListener("keydown", handler);
}, [onClose]);

// 3. Click backdrop (ferme) vs click panneau (ne ferme pas)
<div className="fixed inset-0" onClick={onClose}>
  <div onClick={(e) => e.stopPropagation()}>
```

- `backdrop-blur-sm` : flou sur le fond, indique que le contexte principal est mis en pause
- `max-h-[90vh] overflow-y-auto` : modal scrollable si contenu long (mobile-friendly)
- `aria-label="Fermer"` sur le bouton X

Badges de rang

```
const badges = ["🏆", "🥈", "🥉"];
```

Hiérarchie visuelle immédiate pour les 3 premiers projets sans texte superflu.

10. Section Skills — Carousel horizontal

Fichier : [components/sections/Skills.tsx](#)

Même pattern que Projects (carousel CSS snap + flèches) — cohérence de l'interface.

Structure : chaque carte = une catégorie de compétences (Backend, Frontend, DevOps...) avec des `skill-tag` à l'intérieur.

11. Section Contact — Formulaire accessible

Fichier : [components/sections/Contact.tsx](#)

Composants shadcn/ui

- `Input`, `Textarea`, `Label` : composants headless accessibles (WAI-ARIA)
- Chaque champ a un `id` + `htmlFor` correspondant → clic sur le label focus le champ
- `required` natif HTML + validation email `type="email"`

États visuels du bouton submit

```
// État normal
<svg>→</svg> {t('contact.form.send')}

// État envoi en cours
<svg className="animate-spin" /> {t('contact.form.sending')}

// Disabled + opacité réduite
disabled={sending}
className={clsx("btn-primary", sending && "opacity-70 cursor-not-allowed")}
```

- `clsx` : évite les concaténations de chaînes conditionnelles fragiles
- `animate-spin` Tailwind : spinner CSS pur, cohérent avec le reste du design
- `cursor-not-allowed` : feedback visuel clair pendant l'envoi

Messages de retour traduits

```
{status === "success" && <p className="text-green-400">{t('contact.form.success')}</p>}
{status === "error" && <p className="text-red-400">{t('contact.form.error')}</p>}
```

Disparaissent après 6 secondes (`setTimeout(() => setStatus("idle"), 6000)`).

3 cartes liens (Email / GitHub / LinkedIn)

Pattern "card clickable" :

```
<Link href={`mailto:${profile.email}`} className="card text-center group">
  <div className="group-hover:bg-primary-500/30 transition-colors"> {/* icône */}
```

`group` + `group-hover` : le survol de la carte entière change l'icône → interaction cohérente.

12. Skeleton Loaders (états de chargement)

Fichiers : [Projects.tsx](#) (ligne 260) + [Skills.tsx](#) (ligne 8)

```
function ProjectSkeleton() {
  return (
    <div className="card animate-pulse">
      <div className="h-5 bg-dark-700 rounded w-3/4" /> {/* titre */}
      <div className="h-3 bg-dark-700 rounded w-full" /> {/* description */}
      ...
    </div>
  );
}
```

Pourquoi des skeletons plutôt qu'un spinner ?

- **Réduit la perception du temps d'attente** : l'utilisateur voit la forme du contenu avant qu'il arrive
- **Évite le layout shift (CLS)** : l'espace est déjà réservé
- `animate-pulse` Tailwind : opacité qui pulse entre 100% et 50%, aucun JS nécessaire

- Les blocs reflètent fidèlement la structure réelle (même proportions)

13. Page 404 personnalisée

Fichier : [app/not-found.tsx](#)

"404" avec effet de double couche

```
{/* Texte fantôme (fond) */}
<p className="text-[8rem] font-bold text-dark-800 select-none">404</p>

{/* Texte gradient (avant) – positionné en absolute */}
<p className="absolute inset-0 text-[8rem] font-bold text-transparent bg-clip-text bg-
gradient-to-br from-primary-400 to-primary-600 select-none">
  404
</p>
```

- Deux `<p>` superposés (relative + absolute)
- Le premier fait le fond sombre, le deuxième le gradient coloré
- `select-none` : empêche la sélection accidentelle de ce texte décoratif

14. SEO & Métadonnées

Fichiers : [app/layout.tsx](#), [app/sitemap.ts](#), [app/robots.ts](#), [app/opengraph-image.tsx](#)

Couverture SEO complète

Élément	Implémenté	Fichier
<title> + template	✓	layout.tsx
<meta description>	✓	layout.tsx
Open Graph (LinkedIn, Facebook)	✓	layout.tsx
Twitter Card	✓	layout.tsx
JSON-LD Schema.org (Person)	✓	layout.tsx
hreflang FR-CA / EN-CA	✓	layout.tsx
Sitemap.xml dynamique	✓	sitemap.ts
Robots.txt	✓	robots.ts
OG Image 1200×630	✓	opengraph-image.tsx
Canonical URL	✓	layout.tsx
Vercel Analytics	✓	layout.tsx

OG Image générée dynamiquement

```
// app/opengraph-image.tsx
export const runtime = 'edge';
export const size = { width: 1200, height: 630 };

export default async function Image() {
  return new ImageResponse(<div>...</div>, { ...size });
}
```

Next.js génère l'image PNG automatiquement et la référence dans les meta tags. L'edge runtime la rend ultra-rapide.

15. Accessibilité (a11y)

Focus visible global

```
/* globals.css */
:focus-visible {
  @apply outline-2 outline-offset-2 outline-primary-500;
}
```

Tous les éléments interactifs ont un focus visible cohérent (indigo) — navigation clavier garantie.

aria-labels sur les éléments non-textuels

```
aria-label="Menu de navigation" // bouton hamburger
aria-label="Fermer"           // bouton X modal
aria-label="Projet précédent" // flèche carousel
aria-label="GitHub"           // icône GitHub footer
aria-label={t('hero.scrollDown')} // flèche scroll hero (traduit)
```

aria-expanded sur le menu mobile

```
aria-expanded={isMobileMenuOpen ? "true" : "false"}
```

Note : doit être une **chaîne** en JSX ("true"/"false"), pas un booléen.

aria-hidden sur le backdrop

```
<motion.div aria-hidden="true" /> // backdrop – décoratif, pas interactif
```

Labels formulaire

```
<Label htmlFor="contact-name"> → <Input id="contact-name">
```

Association explicite label-input pour les lecteurs d'écran.

rel="noopener noreferrer" sur les liens externes

```
<Link target="_blank" rel="noopener noreferrer">
```

Sécurité + performance (pas d'accès à `window.open()`).

16. Performance

Scroll event avec cleanup

```
useEffect(() => {  
  const handleScroll = () => setIsScrolled(window.scrollY > 50);  
  window.addEventListener("scroll", handleScroll);  
  return () => window.removeEventListener("scroll", handleScroll);  
}, []);
```

Toujours retirer les event listeners dans le cleanup pour éviter les fuites mémoire.

Police Google Fonts optimisée

```
<!-- layout.tsx -->  
<link rel="preconnect" href="https://fonts.googleapis.com" />  
<link rel="preconnect" href="https://fonts.gstatic.com" crossOrigin="anonymous" />  
<link href="...Inter...&display=swap" rel="stylesheet" />
```

- `preconnect` : établit la connexion TCP/TLS en avance
- `display=swap` : affiche la police système pendant le chargement, puis swap → pas de FOIT (Flash Of Invisible Text)

scroll-behavior: smooth global

```
html { scroll-behavior: smooth; }
```

Navigation intra-page fluide (clic sur les liens de la navbar).

Images : pointer-events-none sur les décorations

Les glows/blobs décoratifs ont `pointer-events:none` → n'interceptent pas les clics, pas de surface invisible cliquable.

IntersectionObserver avec cleanup

```
sections.forEach((s) => observer.observe(s));  
return () => sections.forEach((s) => observer.unobserve(s));
```

17. Internationalisation FR/EN

Fichier : [context/LanguageContext.tsx](#)

- Toutes les chaînes UI passent par `t('clé.sous-clé')`
- Sélecteur de langue dans le header (desktop et mobile)
- Firebase stocke les données en FR et EN séparément
- Les traductions auto via l'API `/api/translate` lors des saves admin

18. Anti-spam formulaire (Honeypot)

Fichier : [components/sections/Contact.tsx](#) + [app/globals.css](#)

```
<div aria-hidden="true" className="honeypot-field">
  <input
    name="website"
    type="text"
    tabIndex={-1}          // ignoré par navigation clavier
    autoComplete="off"    // pas rempli par les questionnaires de mots de passe
  />
</div>
```

```
.honeypot-field {
  position: absolute;
  left: -9999px;          /* hors de l'écran */
  opacity: 0;
  height: 0;
  overflow: hidden;
}
```

Logique :

- Un vrai utilisateur ne voit pas ce champ et ne le remplit jamais
- Les bots remplissent souvent tous les champs → si ce champ est rempli, la requête est rejetée côté serveur
- Combiné avec un timer (`elapsed: Date.now() - mountedAt`) : si le formulaire est soumis en moins de 2-3 secondes, c'est suspect

Résumé des patterns réutilisables

Pattern	Outil	Usage
Animation au scroll	framer-motion useInView	FadeInSection
Hover spring sur carte	motion.whileHover + spring	ProjectCard
Transitions entrée/sortie	AnimatePresence	Menu mobile, ThemeToggle
Dark mode sans flash	next-themes + suppressHydrationWarning	ThemeProvider
Skeleton shimmer	animate-pulse Tailwind	Projects, Skills loading

Accordion CSS pur	<code>max-h transition-all</code>	About "Lire la suite"
Carousel snap natif	<code>snap-x snap-mandatory</code> CSS	Projects, Skills
Modal avec scroll lock	<code>body.style.overflow = "hidden"</code>	ProjectModal, Menu mobile
Fermeture Escape	<code>addEventListener("keydown") + cleanup</code>	Modal, Menu mobile
Focus visible global	<code>:focus-visible</code> outline CSS	globals.css
Honeypot anti-spam	Champ caché hors écran	Contact form
Classes conditionnelles	<code>clsx()</code>	Contact submit button
Icônes légères	<code>lucide-react</code>	Hero, Header

Dernière mise à jour : 2026-02-28